

Abstraction in Java

Abstraction

- Abstraction in Java is the technique of displaying just essential details/functionality to the user. Non-essential implementation details are not shown to the user.
- The television remote control is a great illustration of abstraction. It makes it easier to engage with a TV by hiding the complexity behind simple buttons and symbols, eliminating the need to grasp technical information about how the TV works.
- In Java, abstraction is achieved by **interfaces** and **abstract classes**.

Abstract Classes & Methods

- An abstract class is a class that is declared with an **abstract** keyword.
- An abstract method is a method that is declared **without implementation**.
- **An abstract class may or may not have all abstract methods. Some of them can be concrete methods.**
- A abstract method must always be redefined in the subclass, thus making **overriding compulsory**.
- Any class that contains one or more abstract methods must also be declared with an **abstract** keyword.
- There can be **no object of an abstract class**.
- An abstract class can have parameterized constructors and the default constructor is always present in an abstract class.

Implementing Abstraction in Java

1. Determine the classes or interfaces that will be part of the abstraction.
2. Create an abstract class or interface that defines the common behaviors and properties of these classes.
3. Define abstract methods within the abstract class or interface that do not have any implementation details.
4. Implement concrete classes that extend the abstract class or implement the interface.
5. Override the abstract methods in the concrete classes to provide their specific implementations.
6. Use the concrete classes to contain the program logic.

Example 1: Abstract Class that has Abstract method

```
// Abstract class
abstract class Sunstar {
    abstract void printInfo();
}

// Abstraction performed using extends
class Employee extends Sunstar {
    void printInfo()
    {
        String name = "avinash";
        int age = 21;
        float salary = 222.2F;

        System.out.println(name);
        System.out.println(age);
        System.out.println(salary);
    }
}
```

```
// Base class
class Base {
    public static void main(String args[])
    {
        Sunstar s = new Employee();
        s.printInfo();
    }
}
```

Example 1: Abstract Class that has Abstract method

Output:

```
avinash  
21  
222.2
```

Example 2: Abstract Class having constructor, data member, and methods

```
// Java Program to implement Abstract Class
// having constructor, data member, and methods
import java.io.*;

abstract class Subject {
    Subject() {
        System.out.println("Learning Subject");
    }

    abstract void syllabus();

    void Learn() {
        System.out.println("Preparing Right Now!");
    }
}

class IT extends Subject {
    void syllabus() {
        System.out.println("C , Java , C++");
    }
}
```

```
class GFG {
    public static void main(String[]
args) {
        Subject x = new IT();

        x.syllabus();
        x.Learn();
    }
}
```

Example 2: Abstract Class having constructor, data member, and methods

Output:

```
Learning Subject  
C, Java, C++  
Preparing Right Now!
```


Observation 1: In Java, just like in C++ an instance of an abstract class cannot be created, we can have references to abstract class type though. It is as shown below via the clean Java program.

```
// Java Program to Illustrate
// that an instance of Abstract
// Class can not be created
```

```
// Class 1
// Abstract class
```

```
abstract class Base {
    abstract void fun();
}
```

```
// Class 2
```

```
class Derived extends Base {
    void fun()
    {
        System.out.println("Derived fun()
called");
    }
}
```

```
// Class 3
// Main class
class Main {
```

```
    // Main driver method
```

```
    public static void main(String args[])
    {
```

```
        // Uncommenting the following line will
        cause
```

```
        // compiler error as the line tries to
        create an
```

```
        // instance of abstract class.
```

```
        // Base b = new Base();
```

```
        // We can have references of Base type.
```

```
        Base b = new Derived();
        b.fun();
```

```
    }
```

```
}
```

Observation 1: In Java, just like in C++ an instance of an abstract class cannot be created, we can have references to abstract class type though. It is as shown below via the clean Java program.

Output:

```
Derived fun() called
```

Observation 2: Like C++, an **abstract class** can contain **constructors** in Java. And a constructor of an abstract class is called when an instance of an inherited class is created. It is as shown in the program below as follows:

```
// Java Program to Illustrate Abstract Class Can
contain Constructors
// Class 1
// Abstract class
abstract class Base {

    // Constructor of class 1
    Base()
    {
        // Print statement
        System.out.println("Base Constructor
Called");
    }

    // Abstract method inside class1
    abstract void fun();
}

// Class 2
class Derived extends Base {

    // Constructor of class2
    Derived()
    {
        System.out.println("Derived Constructor
Called");
```

```
    }

    // Method of class2
    void fun()
    {
        System.out.println("Derived fun()
called");
    }
}

// Class 3
// Main class
class GFG {

    // Main driver method
    public static void main(String args[])
    {
        // Creating object of class 2
        // inside main() method
        Derived d = new Derived();
        d.fun();
    }
}
```

Observation 2: Like C++, an **abstract class** can contain **constructors** in Java. And a constructor of an abstract class is called when an instance of an inherited class is created. It is as shown in the program below as follows:

Output:

```
Base Constructor Called  
Derived Constructor Called  
Derived fun() called
```

Observation 3: In Java, we can have *an abstract class without any abstract method*. This allows us to *create classes that cannot be instantiated but can only be inherited*. It is as shown below as follows with help of a clean java program.

```
// Java Program to illustrate Abstract class
// Without any abstract method

// Class 1
// An abstract class without any abstract method
abstract class Base {

    // Demo method. This is not an abstract
    method.
    void fun()
    {
        // Print message if class 1 function is
        called
        System.out.println(
            "Function of Base class is called");
    }
}

// Class 2
class Derived extends Base {
    // This class only inherits the Base class
    methods and
    // properties
}
```

```
// Class 3
class Main {

    // Main driver method
    public static void main(String args[])
    {
        // Creating object of class 2
        Derived d = new Derived();

        // Calling function defined in class 1
        inside main()
        // with object of class 2 inside
        main() method
        d.fun();
    }
}
```

Observation 3: In Java, we can have *an abstract class without any abstract method*. This allows us to *create classes that cannot be instantiated but can only be inherited*. It is as shown below as follows with help of a clean java program.

Output:

Function of base class is called

Observation 4: *Abstract classes can also have **final** methods* (methods that cannot be overridden)

```
// Java Program to Illustrate Abstract classes
// Can also have Final Methods

// Class 1
// Abstract class
abstract class Base {

    final void fun()
    {
        System.out.println("Base fun() called");
    }
}

// Class 2
class Derived extends Base {

}
```

```
// Class 3
// Main class
class GFG {

    // Main driver method
    public static void main(String args[])
    {
        // Creating object of abstract
        class
            Base b = new Derived();
        // Calling method on object
        created above
        // inside main method

        b.fun();
    }
}
```

Observation 4: *Abstract classes can also have **final** methods* (methods that cannot be overridden)

Output:

Base fun() called

Train Ticket Management System

1. Creates a base **abstract class** called **Ticket** with the following:
 - **Abstract methods:**
 - **calculate_price():** Calculates the price of the ticket
 - **display_info():** Displays information about the ticket
 - **Common properties:**
 - **ticket_id:** A unique identifier for the ticket
 - **origin:** The starting station
 - **destination:** The destination station
 - **date:** The date of travel
2. Implement 2 concrete classes that inherit from the **Ticket** abstract class:
 - **StandardTicket:**
 - Additional property: **seat_number, base_price**
 - Implement **calculate_price():** $\text{base_price} + \text{distance_based_fee}$: Take absolute value of the difference of length of origin and destination and multiply by 100.
 - Implement **display_info():** Show all ticket details including seat number
 - **FirstClassTicket:**
 - Additional properties: **seat_number, base_price, first_class_surcharge**
 - Implement **calculate_price():** $\text{Base price} + \text{distance-based fee} + \text{first-class surcharge}$
 - Implement **display_info():** Show all ticket details including seat number

Train Ticket Management System

1. Create a **TicketManager** class that can:
 - Store a collection of tickets: Use **ArrayList** to store ticket objects.
 - Add new tickets: Create a method **add_ticket()** to add newly purchased ticket to the ArrayList.
 - Remove tickets: Create a method **del_ticket()** to remove a ticket from the ArrayList.
 - Find tickets by ticket ID: Create a method **search_info()** to search a ticket information using ticket ID.
 - Calculate total revenue from all tickets: Create a method **total_revenue()** to calculate the total price of purchased ticket using the ArrayList.
2. Implement a simple **command-line interface** to interact with the **TicketManager**, allowing users to:
 - Create new tickets of different types
 - Display ticket information
 - Search for tickets
 - Calculate total revenue

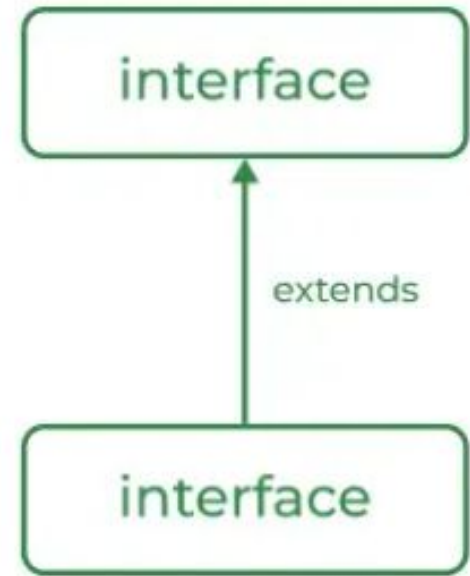
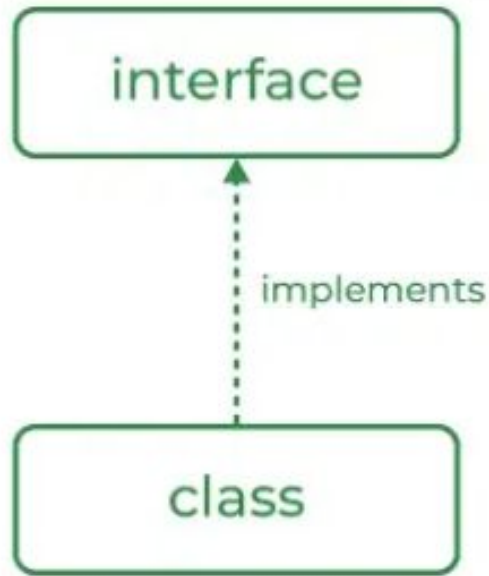
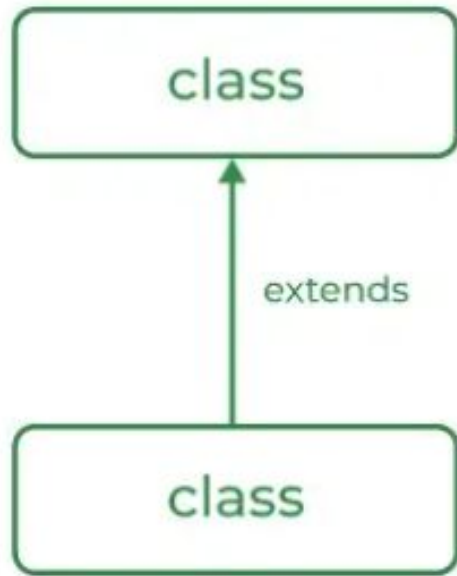
Interface

- Interfaces are another method of implementing abstraction in Java.
- The key difference is that, by using interfaces, we can achieve 100% abstraction in Java classes.
- In Java or any other language, interfaces include both methods and variables but lack a method body.
- o implement an interface, we use the keyword “**implements**” with class.

Abstract Class vs Interface

Points	Abstract Class	Interface
Definition	Cannot be instantiated; contains both abstract (without implementation) and concrete methods (with implementation)	Specifies a set of methods a class must implement; methods are abstract by default.
Implementation Method	Can have both implemented and abstract methods.	Methods are abstract by default; Java 8, can have default and static methods.
Inheritance	class can inherit from only one abstract class.	A class can implement multiple interfaces.
Access Modifiers	Methods and properties can have any access modifier (public, protected, private).	Methods and properties are implicitly public.
Variables	Can have member variables (final, non-final, static, non-static).	Variables are implicitly public, static, and final (constants).

Relationship Between Class and Interface



To implement an interface, we use the keyword implements

```
interface In1 {  
    void display();  
}  
  
class TestClass implements In1 {  
    public void display(){  
        System.out.println("Geek");  
    }  
    public static void main(String[] args)  
    {  
        TestClass t = new TestClass();  
        t.display();  
        System.out.println(t.a);  
    }  
}
```

Consider the example of vehicles like Bike, cars, etc they have common functionalities. So we make an interface and put all these common functionalities. and lets Bike, car, etc implement them in their own class in their own way.

```
interface Vehicle {  
    void changeGear(int a);  
    void speedUp(int a);  
    void applyBrakes(int a);  
}
```

```
class Bike implements Vehicle {  
    int speed;  
    int gear;  
    public void changeGear(int newGear){  
        gear = newGear;  
    }  
    public void speedUp(int increment){  
        speed = speed + increment;  
    }  
    public void applyBrakes(int decrement){  
        speed = speed - decrement;  
    }  
    public void printStates() {  
        System.out.println("speed: " + speed + " gear: " + gear);  
    }  
}
```

Consider the example of vehicles like Bike, cars, etc they have common functionalities. So we make an interface and put all these common functionalities. and lets Bike, car, etc implement them in their own class in their own way.

```
class Main {  
    public static void main (String[] args)  
    {  
        Bike bike = new Bike();  
        bike.changeGear(1);  
        bike.speedUp(4);  
        bike.applyBrakes(3);  
        System.out.println("Bike present  
state :");  
        bike.printStates();  
    }  
}
```


Multiple Inheritance in Java Using Interface

- Multiple Inheritance is an OOPs concept that can't be implemented in Java using classes. But we can use multiple inheritances in Java using Interface.
- Interfaces are non-instantiable and provide a representation of what a class should implement i.e., interfaces do not contain detailed instructions for their behaviors. Because of this feature, a class can implement any number of interfaces without any ambiguity as the implementation is provided by the class itself.
- [Read More From Here](#)

Multiple Inheritance in Java Using Interface

```
interface API {  
    default void show()  
    {  
        System.out.println("Default API");  
    }  
}  
  
interface Interface1 extends API {  
    void display();  
}  
  
interface Interface2 extends API {  
    void print();  
}
```

```
class TestClass implements Interface1, Interface2 {  
    public void display()  
    {  
        System.out.println("Display from Interface1");  
    }  
    public void print()  
    { System.out.println("Print from Interface2"); }  
    public static void main(String args[])  
    {  
        TestClass d = new TestClass();  
        d.show();  
        d.display();  
        d.print();  
    }  
}
```

We can now define static methods in interfaces that can be called independently without an object.

```
interface Student
{
    public void data();
}
class avi implements Student
{
    public void data ()
    {
        String name="avinash";
        int rollno=68;
        System.out.println(name);
        System.out.println(rollno);
    }
}
```

```
public class inter_face
{
    public static void main (String args [])
    {
        avi h= new avi();
        h.data();
    }
}
```

Meal Management System

1. Define an Interface Meal

- Define an **interface** called **Meal** with the following methods:
 - **calculateCost()**: Calculates the cost of the meal.
 - **displayInfo()**: Displays information about the meal.

2. Implement **Concrete** Classes

- Implement the following concrete classes that inherit from the Meal interface:
- **StandardMeal:**
 - Properties: **name, base_price**
 - **Implement calculateCost()**: Returns the price of the meal. (**base_price*1.2**)
 - **Implement displayInfo()**: Displays the name and price of the meal.
- **ComboMeal:**
 - Properties: **name, base_price**
 - **Implement calculateCost()**: Returns the price of the meal. (**base_price*2.5**)
 - **Implement displayInfo()**: Displays the name and price of the meal.
- **DiscountMeal:**
 - Properties: **name, base_price, discountPercentage**
 - **Implement calculateCost()**: Returns the price after applying the discount. (**base_price - base_price*discountPercentage**)
 - **Implement displayInfo()**: Displays the name and price of the meal.

Meal Management System

1. Create a **MealManager** class which should be able to:
 - Store a collection of meals. (**ArrayList**)
 - Add new meals. (**add_meal()**)
 - Remove meals. (**cancel_meal()**)
 - Calculate the total cost of all meals. (**total_meal_revenue()**)
2. Implement a Simple **command-line interface** that allows users to:
 - Add new meals of different types.
 - Display meal information.
 - Calculate the total cost of all meals.

References

- **Java - The Complete Reference by Herbert Schildt**
- <https://www.geeksforgeeks.org/java/>
- <https://www.w3schools.com/java/>
- <https://www.javatpoint.com/java-tutorial>
- <https://www.tutorialspoint.com/java/index.htm>
- <https://www.programiz.com/java-programming>
- **<https://github.com/Rafiquzzaman420/Free-Programming-Books>**